

Demo-Ready Playbook — Battle-Test, Live-Run & Sell Your Other Projects



A project is **demo-ready** when a stranger can watch it *work* in under two minutes and believe it. Two pillars: **battle-tested** (runs on a clean machine, on inputs you didn't hand-pick, without babysitting) and **live-run** (you can execute it end-to-end right now and produce a real, recorded output).

The demo-ready method (repeat for any project)

1. **Name the ONE claim the demo must prove.** One, not five. (“Detects unsafe images on-device in milliseconds.” “Holds a consistent face on a free GPU.”)
2. **Clean-room run.** Fresh environment (new Kaggle / Colab / venv / phone), from the *exact zip you'd ship*. If it breaks here, every buyer hits the same wall — fix it or document it.
3. **Adversarial inputs.** Feed it data you did *not* cherry-pick. Note where it fails. A disclosed limitation beats a hidden one.
4. **Capture proof.** Screen-record the run and save the artifact — output file, metrics, confusion matrix. Numbers beat adjectives.
5. **Write the honest “does NOT do” list** from what you saw in step 3. This is your refund shield.
6. **Package the exact thing you ran** — not a “cleaned-up” version you never tested.

7. **Cut a sub-90-second proof clip:** the one claim → the live run → the result. That clip is what sells.

Your projects, made demo-ready

Project	Prove this one claim	Battle-tested means	Live run means	Honest limit to disclose
SafeScreen (moderation model + SDK)	Flags unsafe images on-device, offline, fast, with real precision/recall	Eval script on a held-out set you did NOT train on → precision, recall, F1, confusion matrix, false-positive rate on safe images	A tiny demo page: upload an image → flagged in ms, offline — recorded	Per-class weakness; only claim the classes where recall is strong
Solana Sniper (study + source)	A rigorous 6-month paper-trading study + hardened architecture + full source	Re-run the paper-trading / backtest harness; show equity curve, win rate, drawdown honestly (losses included)	A paper run on historical data — reproducible, ZERO real funds, secrets stripped	Not financial advice; not a live trader; past results ≠ future
Convergence Framework (kit)	Decide in an hour if a goal is formalizable — before building for months	Apply it to several real objectives, including a deliberately messy / non-formalizable one	A worked example on video: real goal → run the framework → verdict	It's a thinking discipline, not code that runs; the value is the judgment
GODSHAND film kit (done)	Consistent face + film grade on a free GPU	Clean fresh-Kaggle run from the shipped zip	The 4-shot demo film + walkthrough (already speced)	Needs Kaggle + HF token; ~8–9 min/shot; stylized output

🧭 **Do SafeScreen next.** Per your own roadmap it's the cleanest B2B case — recurring buyers, zero legal/financial risk, and the demo is just an eval table + a 60-second upload-and-flag screen recording.

⚠️ Honesty guardrails

- **api_money_bot — do NOT sell or demo.** It's a ToS-violating key harvester. No demo makes that safe; it's a legal tail you can't carry.
- **Solana Sniper — never demo with real money or pitch it as a profit machine.** Paper-trading study only. That framing is both the honest truth and your legal shield.
- **Strip secrets before zipping** every project — remove .env / .pem / key files, add a .env.example. (You already flagged this for Solana.)
- **Always record on a clean machine** so a buyer's first-run experience equals your demo.

Which agent should do the iterating?

Yes — Antigravity (Google's agentic IDE) is a strong, honest fit for you.

Why it fits: its agent operates across the editor, terminal, and browser autonomously — it writes code, runs it, reads the failure, and fixes it, then re-runs. That is exactly the patch → run → fix loop you don't want to do by hand. During the public preview it's free, with access to Gemini 3 Pro and Claude models — the right price for your situation. Reviewers currently rate it the best *free* agentic IDE.

The honest caveats:

- Rate limits are tight — quota refreshes roughly every 5 hours, and even paying users hit walls. Work in focused sessions; don't expect it to grind unattended for hours.
- Costs past the free quota aren't clearly documented yet.
- It runs on **your** machine, so it can iterate locally on the CPU-friendly projects (SafeScreen eval, Convergence, the Solana paper harness). Anything GPU-heavy (the film) still belongs on Kaggle.

If it throttles you, fall back to:

- Cursor (free tier) — polished, same kind of loop.
- Claude Code — predictable flat pricing, excellent at whole-repo work.
- aider or OpenCode / Codex CLI with your own API key — unlimited, most frugal.

Bottom line: start with Antigravity free on SafeScreen. If rate limits interrupt you mid-loop, switch to Cursor free or aider.

Ready-to-paste agent prompts

Paste the **master prompt** first, then paste the matching **project block** right under it in the same message.

Master prompt (use for every project)

```
You are my senior engineer. Your goal: make this project DEMO-READY, which means battle-t
ested + live-run.

Context: I sell small digital products. This must run for a stranger on a clean machine a
nd produce ONE believable proof in under 2 minutes.

The ONE claim this demo must prove: <FILL IN>

Work this loop autonomously until it is done. Do NOT stop at the first error to ask me —
keep iterating:
1. Create a fresh virtual environment and install from the exact files in this project. L
ist every dependency you install.
2. Run the entry point / test / eval command. If it fails, read the real error, fix the c
ode, and re-run. Repeat until it runs clean.
3. Feed it inputs I did NOT cherry-pick (edge cases, empty, malformed, oversized). Log ex
actly what breaks.
4. Produce a real artifact as proof: an output file plus metrics (numbers, not adjective
s). Save it to disk.
5. Write an honest "what it does NOT do" list from the failures you saw in step 3. Do not
hide them.
6. Strip all secrets (.env, .pem, keys, tokens). Add a .env.example. Repackage the exact
tested version into a clean zip.
7. Write a 6-line README: what it is, the one claim, how to run it in 3 steps, known limi
ts.

Hard rules:
- Never fake, mock, or hardcode results. If a number is bad, show the real number.
- Never invent capabilities. Only claim what you actually ran.
```

```
- Print every command you ran so I can reproduce it on camera.
- Finish by giving me a sub-90-second demo script: the claim -> the command -> the resul
t.
```

SafeScreen (moderation model + SDK)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Flags unsafe images (violence / gore / NSFW) on-device, offline, in ~30-5
0ms, with real precision and recall."

Specifics:
- The model is toxic_detector.onnx; run it with onnxruntime on CPU.
- First, confirm the model loads and classifies ONE image end-to-end before anything els
e.
- Build a held-out test set I did NOT train on. Report precision, recall, F1, a confusion
matrix, and the false-positive rate on safe images — PER CLASS.
- Only keep marketing claims for classes where recall is actually strong. Flag weak class
es as limits.
- Live-run demo: a minimal FastAPI or Gradio page — upload an image, get back the flagged
label + confidence, fully offline. Record it.
```

Solana Sniper (paper-trading study + source)

```
[Paste the master prompt above first, then this:]

The ONE claim: "A rigorous 6-month paper-trading STUDY with a hardened architecture and f
ull source — NOT a live money-maker."

Specifics:
- Run ONLY in paper-trading / backtest mode on historical data. NEVER connect real funds,
real keys, or mainnet execution. This is a hard safety rule.
- Re-run the Monte Carlo / backtest harness. Report the equity curve, win rate, and max d
rawdown, and show losing runs honestly.
- Strip every secret and private key. Add .env.example. Verify no wallet or key is left i
n the zip.
- The docs must state, verbatim: not financial advice; paper-tested only; live performanc
e unverified; past results do not predict future results.
- Live-run demo: reproduce the study's output from the shipped zip with a single command.
```

Convergence Framework (kit)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Decide in under an hour whether a goal is formalizable — before you build
for months."

Specifics:
- This is a thinking framework + system prompt, not a long-running program. "Battle-test"
here means: apply it to 3-4 real objectives, including one deliberately messy / non-forma
lizable goal, and show it correctly flags the messy one.
- Produce a worked-example doc for each: real goal -> framework applied -> verdict.
- Package the system prompt + kit + the worked examples together.
- Be honest in the README: the value is the judgment and discipline, not executable code.
```

Where to sell + who buys

Creators — for the film kit

(Full targeting lives on the GODSHAND listing page; the joinable servers:)

- YouTube Faceless (~20k) — huge, but **strictly no self-promo**; give value first.
- Faceless Incubator (~38k) — largest faceless-creator hub.
- Faceless Success — free community, story-sharing friendly.
- Wan (official, ~6.6k) — the exact model your kit uses.
- AI HUB (~21k) — broad create/learn/collaborate server.
- Reddit: r/aitubers, r/aifilmmaking, r/comfyui, r/StableDiffusion, r/PartneredYoutube.

Developers — for SafeScreen, Convergence, Solana study

Different buyers entirely (they don't hang out with faceless YouTubers):

- Reddit: r/androiddev, r/iOSProgramming, r/computervision, r/MachineLearning, r/SideProject; r/algotrading (Solana *study* only — respect their no-promo/no-signal rules).
- **Indie Hackers** and **Hacker News "Show HN"** — ideal for a battle-tested dev tool with real metrics.

- Mobile-ML / computer-vision Discords for SafeScreen; agent-builder communities for Convergence.

India-specific

- Indian AI Production — Telegram + linked Discord, active AI-video crowd.
- **AI Learners India** — Hindi, no-code AI community; friendly to beginners on a budget.
- Heads-up: Facebook “low-budget Indian AI video creator needed” posts (~₹100/video) are *gig-hire* leads, a different market — but they're warm prospects who might buy the kit instead of hiring out.

Selling-platform reality

- **No free discovery.** Gumroad / Payhip won't send you cold traffic — you bring your own from posts and clips.
- **Start price low, scale with proof.** Don't launch high with zero sales.
- **Clean, professional cover images** — don't let it look AI-thrown-together.
- **Fees:** Payhip ~5%, Gumroad ~10%. Factor it in.
- **People buy fixes to small annoying problems**, not grand solutions. Frame each demo around the one irritation it removes.

The film kit is your template. Same seven steps, per project — battle-test, live-run, record the proof, package the exact thing, sell the outcome.